



---

# Εκπαιδευτικός Σχεδιασμός STEM

---

| <i>ΟΝΟΜΑΤΕΠΩΝΥΜΟ</i> | <i>ΑΡΙΘΜΟΣ<br/>ΜΗΤΡΩΟΥ</i> | <i>EMAIL</i>          | <i>ΗΜΕΡΟΜΗΝΙΑ</i> |
|----------------------|----------------------------|-----------------------|-------------------|
| ELVIS AVDULI         | E22001                     | avdulielvis@gmail.com | 16/11/2025        |

**7<sup>ο</sup> Εξάμηνο**



## Πίνακας περιεχομένων

|                                                    |    |
|----------------------------------------------------|----|
| 1 <sup>η</sup> Εργασία: Scratch .....              | 2  |
| 1.1 Εκφώνηση .....                                 | 2  |
| 1.3 Οδηγίες Παιχνιδιού .....                       | 3  |
| 1.3 Τεχνική Αναφορά .....                          | 5  |
| 1.3.1 Car .....                                    | 5  |
| 1.3.2 Traffic_Light_Controller .....               | 10 |
| 1.3.3 Stop_NS & Stop_EW.....                       | 11 |
| 1.3.4 Traffic Lights (N-S-W-E) .....               | 12 |
| 1.3.5 Start_Button .....                           | 13 |
| 1.3.6 End_Screen_Display .....                     | 13 |
| 1.3.7 Stages.....                                  | 14 |
| 2 <sup>η</sup> εργασία: Makecode .....             | 15 |
| 2.1 Εκφώνηση .....                                 | 15 |
| 2.2 Οδηγίες Χρήσης .....                           | 16 |
| 2.3 Τεχνική Αναφορά .....                          | 16 |
| 2.3.1 Αρχικοποίηση Παραμέτρων .....                | 17 |
| 2.3.2 Κεντρικός Βρόχος και Ροή Παιχνιδιού .....    | 17 |
| 2.3.3 Αλγόριθμος Ελέγχου Επίλυσης Λαβυρίνθου ..... | 18 |
| 2.3.4 Απεικόνιση Παιχνιδιού.....                   | 19 |
| 2.3.5 Συνάρτηση Εκτέλεσης Επιπέδου .....           | 20 |
| 3 <sup>η</sup> εργασία: TinkerCad .....            | 22 |
| 3.1 Λυση .....                                     | 22 |
| 3.2 Εκφώνηση .....                                 | 23 |
| 3.3 Οδηγίες Χρήσης .....                           | 23 |
| 3.4 Συνδεσμολογία .....                            | 25 |
| 3.5 Τεχνική Αναφορά .....                          | 27 |

# 1<sup>η</sup> Εργασία: Scratch

## 1.1 Εκφώνηση

“Σας ζητείται η δημιουργία ενός παιχνιδιού στο Scratch με τίτλο «Εξυπνη Πόλη – Διαχείριση Κυκλοφορίας». Ο παίκτης έχει ως αποστολή να ελέγχει τα φανάρια μιας πολυσύχναστης διασταύρωσης με στόχο να διευκολύνει την κυκλοφορία των οχημάτων και να αποφύγει τις συγκρούσεις. Από τα διάφορα σημεία της οθόνης θα εμφανίζονται αυτοκίνητα με τυχαίο βαθμό τα οποία θα κινούνται προς τη διασταύρωση. Ο παίκτης θα έχει την επιλογή είτε να αλλάζει τα φανάρια χρησιμοποιώντας συγκεκριμένα πλήκτρα του πληκτρολογίου είτε να αναπτύξει αλγόριθμο όπου με βάση την κίνηση θα αλλάζουν οι καταστάσεις των φαναριών, έτσι ώστε να επιτρέπει την ασφαλή διέλευση των οχημάτων. Με την πάροδο του χρόνου η δυσκολία θα αυξάνεται, καθώς θα εμφανίζονται περισσότερα οχήματα σε μικρότερο χρονικό διάστημα και η πιθανότητα δημιουργίας κυκλοφοριακής συμφόρησης θα μεγαλώνει.

Η διασταύρωση θα έχει δύο κατευθύνσεις Βόρεια - Νότια και Ανατολική - Δυτική, ενώ έκαστη θα αποτελείται από δύο λωρίδες. Για κάθε όχημα που περνά με ασφάλεια από τη διασταύρωση θα προστίθεται ένας πόντος, ενώ σε περίπτωση σύγκρουσης θα αφαιρείται μία ζωή. Επίσης, στις διασταυρώσεις θα διατηρούνται στατιστικά όπως ο μέγιστος και μέσος χρόνος αναμονής, ο ρυθμός διέλευσης, ο αριθμός των αυτοκινήτων ανά κατεύθυνση και οι συγκρούσεις.

Το παιχνίδι ξεκινά με τρεις ζωές και τερματίζεται όταν αυτές εξαντληθούν. Θα συμπεριλαμβάνει μια αρχική οθόνη με οδηγίες χρήσης, τη δυνατότητα στο χρήστη να επιλέξει μεταξύ χειροκίνητου και αυτόματου τρόπου λειτουργίας των φαναριών, καθώς και μια τελική οθόνη που θα εμφανίζει το τελικό σκορ του παίκτη, τον αριθμό των συγκρούσεων και τη μέση αναμονή.

Απαιτείται να χρησιμοποιηθούν ελκυστικά οπτικά στοιχεία (sprites) για τα οχήματα, τους δρόμους και τα φανάρια, καθώς και ηχητικά εφέ για επιτυχημένες διελεύσεις και συγκρούσεις, προκειμένου το αποτέλεσμα να είναι πιο διαδραστικό και ρεαλιστικό.”

## 1.3 Οδηγίες Παιχνιδιού

### 1. Έναρξη Παιχνιδιού

Στην αρχική οθόνη θα εμφανιστεί το κουμπί *Start* μαζί με τις οδηγίες χρήσης. Ο παίκτης πρέπει να πατήσει το κουμπί για να ξεκινήσει η προσομοίωση της διασταύρωσης και να αρχίσουν να εμφανίζονται οχήματα από όλες τις κατευθύνσεις.

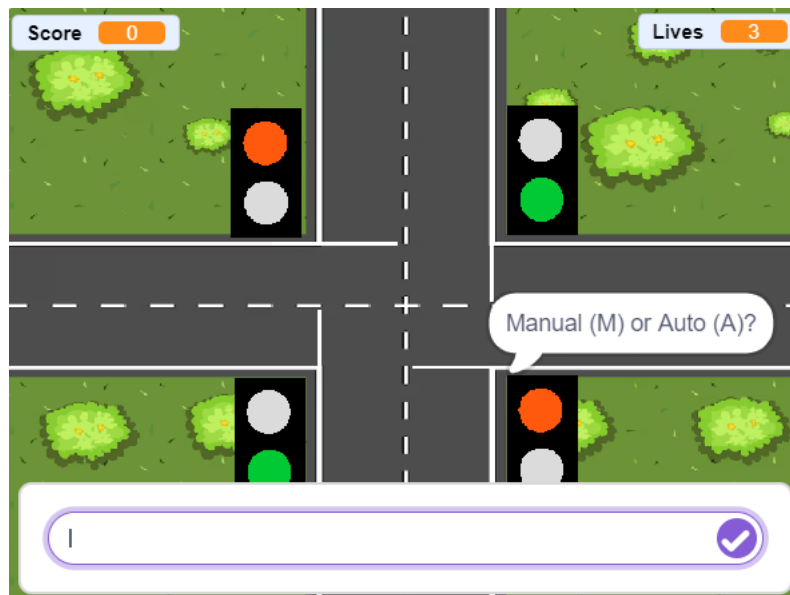


### 2. Επιλογή Τρόπου Ελέγχου Φαναριών

Πριν ξεκινήσει η κυκλοφορία, ο χρήστης πρέπει να διαλέξει ανάμεσα σε δύο τρόπους λειτουργίας:

- **Χειροκίνητος έλεγχος:** Αλλάζεις τα φανάρια πατώντας οπουδήποτε στην οθόνη του παιχνιδιού με το αριστερό κλικ.

- **Αυτόματος έλεγχος:** Τα φανάρια αλλάζουν μόνα τους με βάση έναν αλγόριθμο που ρυθμίζει τη σειρά και τον χρόνο της κάθε κατεύθυνσης.



### 3. Στόχος του Παιχνιδιού

Σκοπός είναι ο παίκτης να επιτρέψει στα αυτοκίνητα να περνούν με ασφάλεια από τη διασταύρωση, αποφεύγοντας τις συγκρούσεις. Κάθε ασφαλής διέλευση προσθέτει πόντους. Αν γίνει σύγκρουση, χάνεται μία ζωή. Το παιχνίδι ξεκινά με τρεις ζωές.

### 4. Αύξηση Δυσκολίας

Καθώς περνάει ο χρόνος, περισσότερα αυτοκίνητα εμφανίζονται πιο γρήγορα και από περισσότερες κατευθύνσεις. Χρειάζεται προσοχή και σωστή χρονική διαχείριση.

## 5. Τερματισμός Παιχνιδιού

Όταν οι ζωές μηδενιστούν, εμφανίζεται η τελική οθόνη με το συνολικό σκορ, τον αριθμό των συγκρούσεων και τον μέσο χρόνο αναμονής των οχημάτων.



## 1.3 Τεχνική Αναφορά

Για την δημιουργία του παιχνιδιού χρησιμοποιήθηκαν 10 sprites:

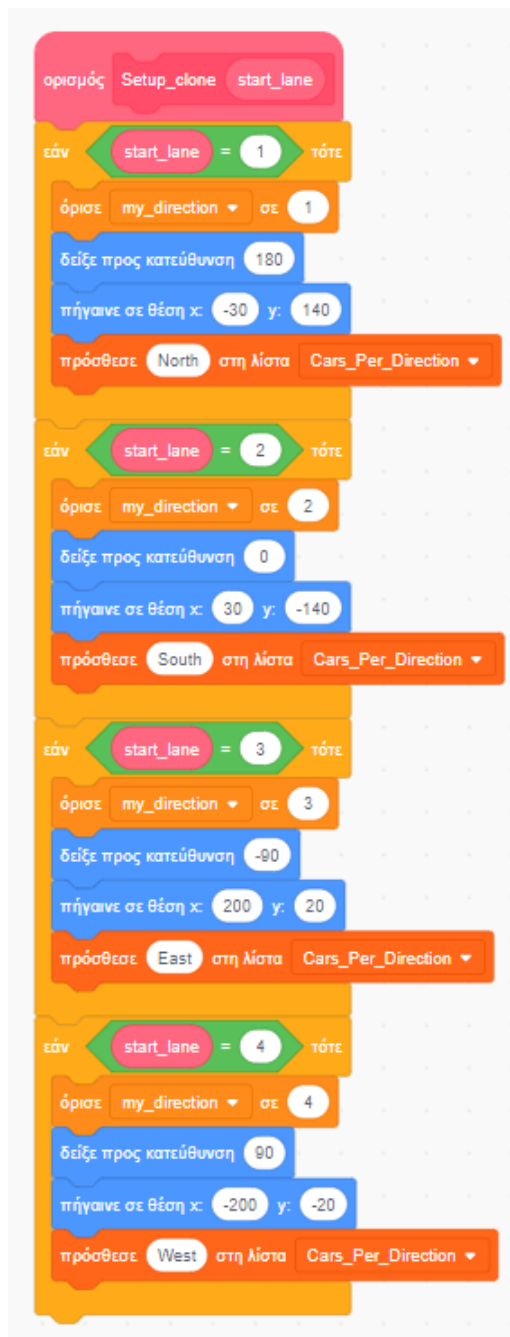
1. Car
2. Traffic\_Light\_Controller (Αόρατο sprite)
3. Stop\_NS
4. Stop\_EW
5. Start\_Button
6. End\_Screen\_Display (Αόρατο sprite)
7. N\_Traffic
8. S\_Traffic
9. W\_Traffic
10. E\_Traffic

### 1.3.1 Car

Το sprite του αυτοκινήτου είναι ένα απο τα 2 βασικότερα Sprites. Με αυτό συντονίζονται και δημιουργούνται τα αυτοκίνητα.

Με το πάτημα της πράσινης σημαίας, γίνεται η αρχικοποίηση: ορίζουμε το **spawn\_rate** σε 10 (ώστε στην αρχή ο ρυθμός εμφάνισης των οχημάτων να είναι 10 δευτερόλεπτα) και τις **Lives** (ζωές) σε 3. Ύστερα, αρχικοποιούνται οι μεταβλητές **Total Collisions** και **Τελος Παιχνιδιου** παίρνοντας την τιμή 0. Στη συνέχεια, σιγουρευόμαστε ότι και οι τρεις λίστες (**Wait\_Times**, **Collisions**, **Cars\_Per\_Direction**) είναι άδειες στην αρχή του παιχνιδιού, διαγράφοντας όλα τα στοιχεία τους. Στο τέλος, μηδενίζεται και το χρονόμετρο.





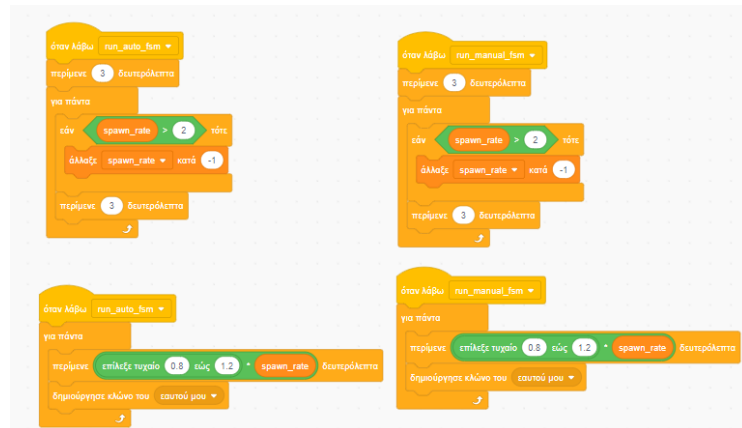
Όταν καλείται το custom block **Setup\_clone**, ανάλογα με τον αριθμό (*start\_lane*) που θα του δώθει, ρυθμίζει τον κλώνο (το όχημα) για να ξεκινήσει από διαφορετική λωρίδα.

Αν το *start\_lane* είναι 1, το όχημα ξεκινά από πάνω (x: -30, y: 140) με κατεύθυνση 180 (προς τα κάτω) και καταγράφεται ως "North". Αν είναι 2, ξεκινά από κάτω (x: 30, y: -140) με κατεύθυνση 0 (προς τα πάνω) και καταγράφεται ως "South". Αντίστοιχα, αν είναι 3, ξεκινά από δεξιά (x: 200, y: 20) με κατεύθυνση -90 (αριστερά) ως "East", και αν είναι 4, ξεκινά από αριστερά (x: -200, y: -20) με κατεύθυνση 90 (δεξιά) ως "West". Κάθε φορά, ορίζεται και η αντίστοιχη τιμή (1, 2, 3 ή 4) στη μεταβλητή *my\_direction*.

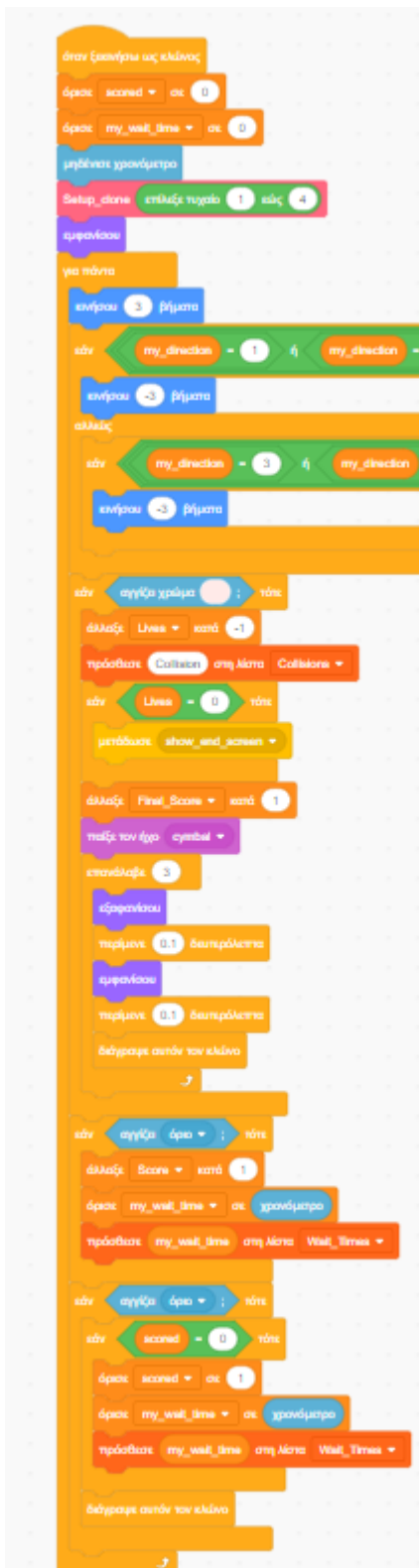
Με αυτόν τον τρόπο, ρυθμίζουμε τα αυτοκίνητα ώστε να ξεκινούν από τη σωστή θέση πάνω στον δρόμο και φροντίζουμε, ανάλογα με τη λωρίδα τους, να έχουν εξ αρχής την κατάλληλη κατεύθυνση για να κινηθούν.



Αυτά τα μπλοκ ελέγχουν πώς δημιουργούνται τα οχήματα και πώς αυξάνεται η δυσκολία του παιχνιδιού, τόσο όταν λαμβάνεται το μήνυμα `run_auto_fsm` (αυτόματη λειτουργία) όσο και το `run_manual_fsm` (χειροκίνητη λειτουργία).



Και στις δύο περιπτώσεις, είτε με το μήνυμα `run_auto_fsm` είτε με το `run_manual_fsm`, η λογική είναι ακριβώς η ίδια. Υπάρχει ένας μηχανισμός αύξησης δυσκολίας: μετά από μια αρχική αναμονή 3 δευτερολέπτων, το παιχνίδι ελέγχει κάθε 3 δευτερόλεπτα αν το `spawn_rate` είναι μεγαλύτερο από 2. Αν είναι, το μειώνει κατά 1. Αυτό σημαίνει ότι όσο περνάει η ώρα, τα οχήματα εμφανίζονται όλο και πιο γρήγορα, μέχρι ο ρυθμός να φτάσει στο κατώτατο όριο (2). Ταυτόχρονα, ένας άλλος βρόχος δημιουργεί συνεχώς τους κλώνους (τα νέα οχήματα). Ο χρόνος αναμονής πριν τη δημιουργία κάθε οχήματος δεν είναι σταθερός, αλλά υπολογίζεται πολλαπλασιάζοντας την τρέχουσα τιμή του `spawn_rate` με έναν τυχαίο αριθμό μεταξύ 0.8 και 1.2. Αυτό κάνει την εμφάνιση των οχημάτων ελαφρώς απρόβλεπτη.



Αυτό το μπλοκ κώδικα εξηγεί τι κάνει κάθε κλώνος-όχημα από τη στιγμή που δημιουργείται μέχρι που φεύγει από την οθόνη.

Όταν ξεκινάει ένας κλώνος, αρχικά μηδενίζει τις τοπικές μεταβλητές του (scored και my\_wait\_time) και το χρονόμετρο. Αμέσως μετά, καλεί τη συνάρτηση Setup\_clone δίνοντας έναν τυχαίο αριθμό από 1 έως 4, για να οριστεί η λωρίδα, η θέση και η κατεύθυνση εκκίνησής του, και έπειτα εμφανίζεται.

Σε έναν συνεχή βρόχο, ο κλώνος κινείται 3 βήματα. Ωστόσο, εάν πλησιάσει σε κόκκινο φανάρι (π.χ. αν η κατεύθυνσή του είναι 1 ή 2, αγγίζει τη γραμμή Stop\_NS και η μεταβλητή NS\_State είναι "Red"), τότε κινείται -3 βήματα, με αποτέλεσμα να σταματάει. Η ίδια λογική ισχύει και για τις κατευθύνσεις 3 και 4 με τη γραμμή Stop\_EW.

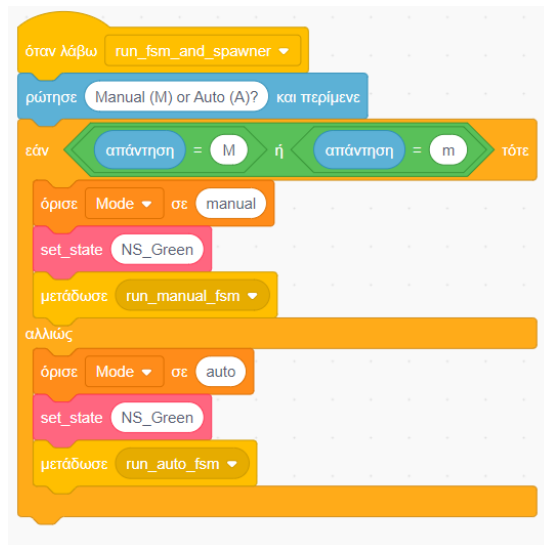
**Σε περίπτωση Σύγκρουσης:** Αν ο κλώνος αγγίξει το χρώμα ενός άλλου οχήματος, ο παίκτης χάνει μια ζωή (Lives - 1) και η σύγκρουση καταγράφεται στη λίστα Collisions.

**Σε περίπτωση Game Over:** Αν οι ζωές μηδενιστούν, ο κλώνος που έκανε τη σύγκρουση στέλνει το μήνυμα show\_end\_screen, παίζει έναν ήχο, αναβοσβήνει μερικές φορές και διαγράφεται.

**Σε περίπτωση Επιτυχίας:** Αν ο κλώνος φτάσει στο όριο (στην άκρη) της πίστας και δεν έχει ήδη σκοράρει (scored = 0), τότε προσθέτει 1 στο **Score**, θέτει το scored σε 1 (για να μην ξανασκοράρει), καταγράφει τον χρόνο του (χρονόμετρο) στη λίστα Wait\_Times και διαγράφεται.

### 1.3.2 Traffic\_Light\_Controller

Το sprite αυτό αν και δεν έχει κάποια ενδυμασία είναι πολύ σημαντικό στην διαχείριση των φαναριών.



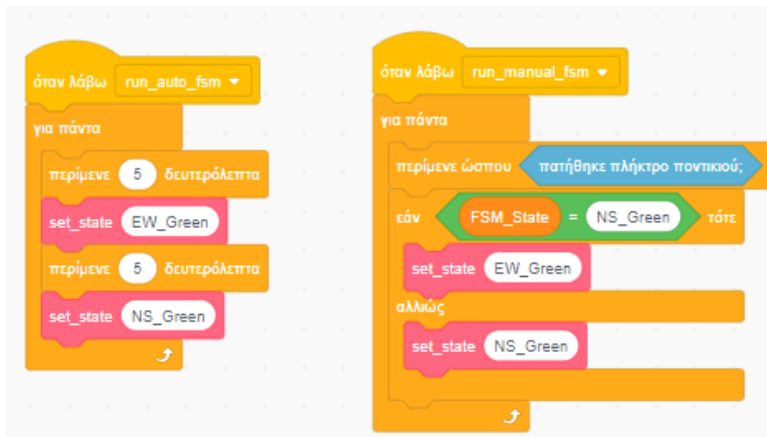
Όταν ληφθεί το μήνυμα `run_fsm_and_spawner`, το πρόγραμμα ρωτά τον παίκτη αν προτιμά χειροκίνητη (Manual - M) ή αυτόματη (Auto - A) λειτουργία. Εάν η απάντηση είναι "M" ή "m", ορίζει τη λειτουργία (Mode) σε `manual` και στέλνει το μήνυμα `run_manual_fsm`. Σε κάθε άλλη περίπτωση, ορίζει τη λειτουργία σε `auto` και στέλνει το μήνυμα `run_auto_fsm`. Και στις δύο περιπτώσεις, ξεκινά καλώντας τη συνάρτηση `set_state` για να κάνει το φανάρι Βορρά-Νότου (NS\_Green) πράσινο

Αυτός είναι το custom block `set_state`, που οποία ελέγχει τα φανάρια. Όταν καλείται, δέχεται μια παράμετρο (`new_state`) που δείχνει ποιο φανάρι πρέπει να ανάψει.

Αν η παράμετρος είναι **NS\_Green** (Βορράς-Νότος), τότε ορίζει το `NS_state` σε "green", το `EW_State` σε "red" και αλλάζει την ενδυμασία του αντικειμένου σε `NS_Green`.

Αντίστοιχα, αν είναι **EW\_Green** (Ανατολή-Δύση), ορίζει το `NS_state` σε "Red", το `EW_State` σε "Green" και αλλάζει την ενδυμασία σε `EW_Green`.

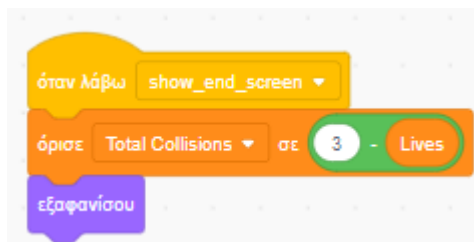




Αυτα δύο μπλοκ ελέγχουν τις δύο διαφορετικές λειτουργίες του παιχνιδιού:

**Αυτόματη Λειτουργία (run\_auto\_fsm):** Όταν ξεκινά αυτή η λειτουργία, το πρόγραμμα μπαίνει σε έναν ατέρμονο βρόχο. Απλώς περιμένει 5 δευτερόλεπτα και καλεί τη συνάρτηση set\_state για να ανάψει το φανάρι EW\_Green, έπειτα περιμένει άλλα 5 δευτερόλεπτα και ανάβει το NS\_Green. Αυτό επαναλαμβάνεται συνεχώς.

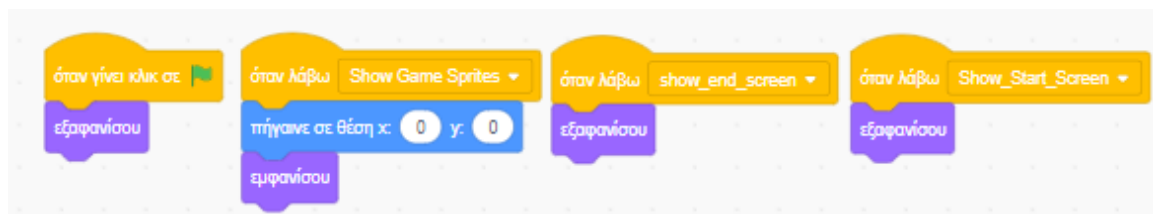
**Χειροκίνητη Λειτουργία (run\_manual\_fsm):** Σε αυτή τη λειτουργία, το πρόγραμμα περιμένει μέχρι ο παίκτης να πατήσει το ποντίκι. Μόλις πατηθεί, ελέγχει την τρέχουσα κατάσταση (FSM\_State): αν είναι NS\_Green το αλλάζει σε EW\_Green, αλλιώς το αλλάζει σε NS\_Green.



Τέλος, όταν χάσει ο παίκτης, ενημερώνεται η μεταβλητή Total Collisions ώστε να εμφανιστούν οι συνολικές συγκρούσεις

### 1.3.3 Stop\_NS & Stop\_EW

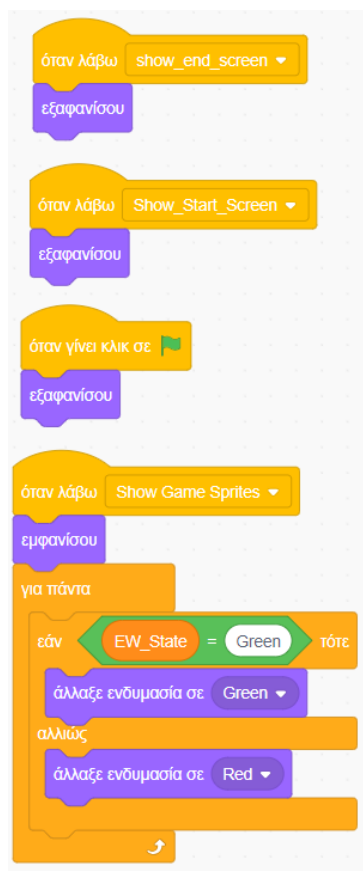
Αυτά τα δύο sprites λειτουργούν ως βοηθητικές λευκές γραμμές stop που καθοδηγούν τα αυτοκίνητα να σταματούν όταν το φανάρι είναι κόκκινο. Το ένα sprite αντιστοιχεί στις βόρεια-νότια κατευθύνσεις και το άλλο στις ανατολικά-δυτικά. Ο κώδικας αυτών των sprites ελέγχει πότε εμφανίζονται, και όταν είναι ορατά, ρυθμίζει και τη θέση τους πάνω στο οδόστρωμα.



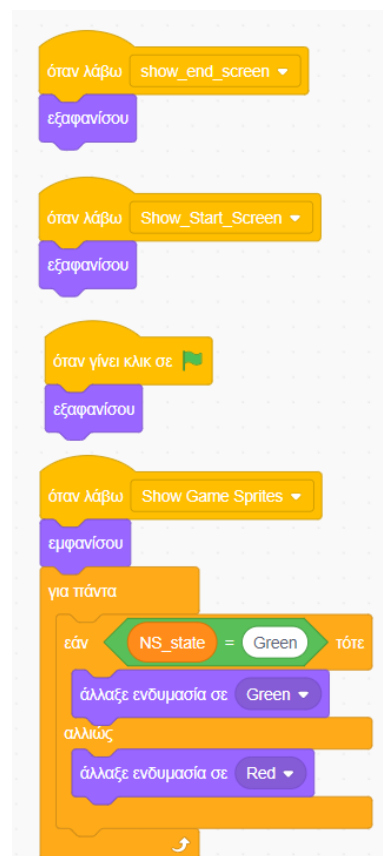
### 1.3.4 Traffic Lights (N-S-W-E)

Τα 4 sprites (ένα για κάθε κατεύθυνση) έχουν από 2 ενδυμασίες red και green και είναι αυτά που αλλάζουν χρώμα τα φανάρια και δείχνουν πότε ένα αμάξι θα σταματήσει. Ελέγχουν αν η αντίστοιχη μεταβλητή είναι Red η Green και αλλάζει κατάλληλα το χρώμα.

N-Traffic & S-Traffic

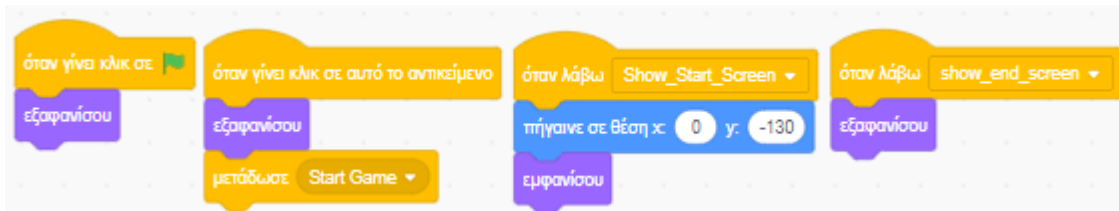


W-Traffic & E-Traffic



### 1.3.5 Start\_Button

Το Start\_Button χρησιμοποιείται για να ξεκινήσει το παιχνίδι, αφού ο παίκτης το πατήσει. Ομοίως με τα Stop\_NS και Stop\_EW, ο κώδικας ελέγχει τότε εμφανίζεται και όταν είναι ορατό, ρυθμίζει τη θέση του πάνω στην αρχική οθόνη.



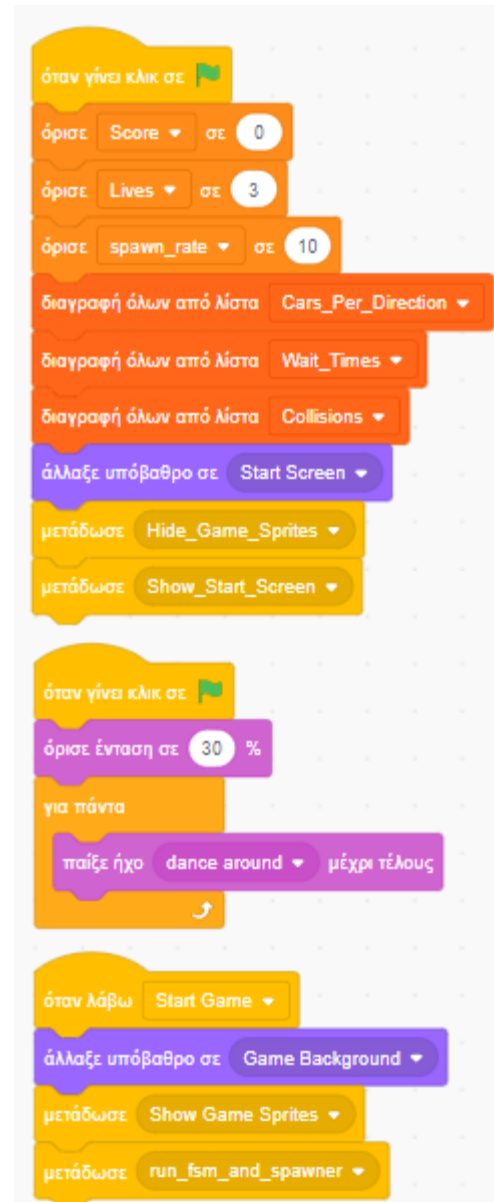
### 1.3.6 End\_Screen\_Display

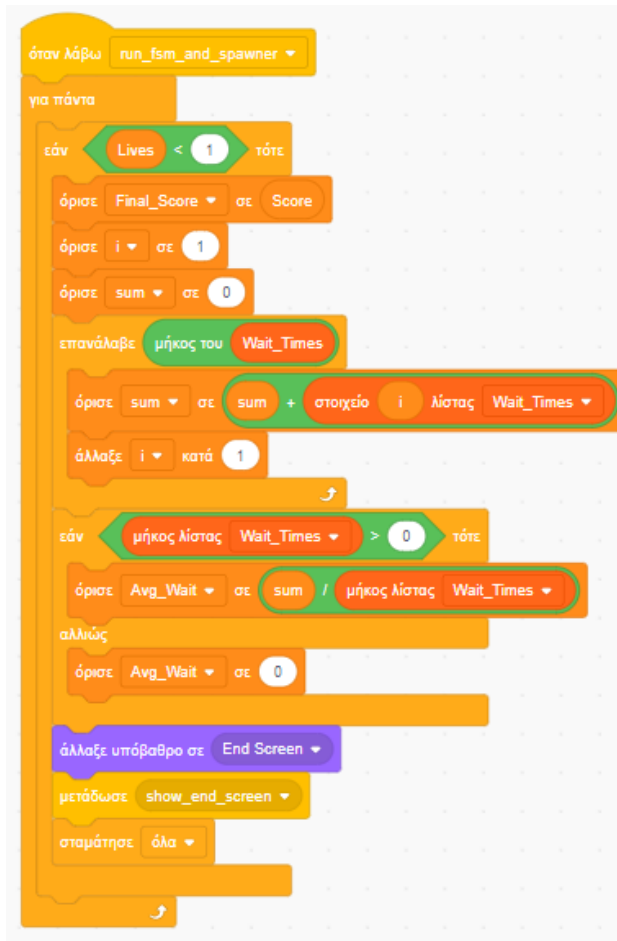
Το sprite End\_Screen\_Display δεν έχει εμφάνιση όμως βοηθάει στην διαχείριση ποιων μεταβλητών θα εμφανίζονται και τότε κυρίως των μεταβλητών που θα εμφανίζουν τα στατιστικά όταν χάνει ο παίκτης.



### 1.3.7 Stages

Όταν πατηθεί η πράσινη σημαία, το παιχνίδι ξανααρχικοποιεί όπως και στο sprite car (για σιγουρια) τις μεταβλητές (Score, Lives, spawn\_rate) και αδειάζει τις λίστες, ενώ ταυτόχρονα ρυθμίζει την ένταση και ξεκινά να παίζει τη μουσική "dance around" συνεχώς. Παράλληλα, αλλάζει το υπόβαθρο στην αρχική οθόνη (Start Screen) και στέλνει μηνύματα για να εμφανιστεί αυτή η οθόνη. Όταν το παιχνίδι λάβει το μήνυμα Start Game (που πιθανότατα στέλνεται από ένα κουμπί), αλλάζει το υπόβαθρο στο κυρίως παιχνίδι (Game Background), εμφανίζει τα αντικείμενα του παιχνιδιού και στέλνει το μήνυμα run\_fsm\_and\_spawner για να ξεκινήσει η κανονική ροή.





Αφού ξεκινήσει η ροή του παιχνιδιού (με το μήνυμα `run_fsm_and_spawner`), ένας κώδικας εκτελείται συνεχώς για να ελέγχει αν οι ζωές (Lives) έπεσαν κάτω από 1. Μόλις αυτό συμβεί, ενεργοποιείται η διαδικασία τέλους παιχνιδιού: καταγράφεται το `Final_Score`, υπολογίζεται το άθροισμα (`sum`) και έπειτα ο μέσος όρος (`Avg_Wait`) όλων των χρόνων αναμονής από τη λίστα `Wait_Times`. Αμέσως μετά, το υπόβαθρο αλλάζει στην οθόνη τέλους (End Screen), στέλνεται το μήνυμα `show_end_screen` και το παιχνίδι σταματά πλήρως.

## 2<sup>η</sup> εργασία: Makecode

### 2.1 Εκφώνηση

“Σας ζητείται η δημιουργία ενός διαδραστικού παιχνιδιού στο *Micro:bit* με τίτλο “Λαβύρινθος”. Στόχος είναι να μετακινήσετε έναν φωτεινό δείκτη μέσα από έναν λαβύρινθο που εμφανίζεται στο πλέγμα LED του *Micro:bit* και να φτάσει στην έξοδο μέσα σε συγκεκριμένο χρονικό όριο. Ο λαβύρινθος σε κάθε επίπεδο θα δημιουργείται τυχαία και θα πρέπει να έχει μια διαδρομή με αρχή και τέλος. Η δυσκολία θα αυξάνεται σταδιακά, είτε δυσκολεύοντας τη διαδρομή είτε συντομεύοντας το διαθέσιμο χρόνο. Ο χρήστης θα ελέγχει το φωτεινό δείκτη με τη λειτουργία *tilt* του *Micro:bit* όπου ανάλογα με την κλίση ο φωτεινός δείκτης θα κινείται (δεξιά/αριστερά/μπροστά/πίσω). Η κίνηση του φωτεινού δείκτη θα



*επιτρέπεται μονάχα στον κενό χώρο ενώ στην περίπτωση σύγκρουσης με τον τοίχο του λαβύρινθου θα δίνεται σχετική ακουστική ανατροφοδότηση. Παρομοίως, η κίνηση του φωτεινού δείκτη θα συνοδεύεται από κατάλληλη ηχητική ανατροφοδότηση. Κατά την έναρξη του παιχνιδιού θα εμφανίζεται μήνυμα εκκίνησης στην οθόνη LED. Ο παίκτης θα ξεκινά πάντα από μία προκαθορισμένη θέση (π.χ. πάνω αριστερή γωνία) και η έξοδος θα βρίσκεται σε αντίθετη θέση (π.χ. κάτω δεξιά). Αν ο παίκτης δεν φτάσει στην έξοδο πριν τη λήξη του χρόνου, το παιχνίδι τερματίζει με σχετική ένδειξη αποτυχίας και αναπαραγωγής κάποιου ήχου.”*

## 2.2 Οδηγίες Χρήσης

Η λειτουργία του παιχνιδιού αξιοποιεί πλήρως τον αισθητήρα κλίσης (tilt) της πλακέτας micro:bit. Για να ξεκινήσει η πρόκληση, ο παίκτης πρέπει πρώτα να πατήσει το κουμπί A. Με το πάτημα, ενεργοποιείται αμέσως η αντίστροφη μέτρηση του χρόνου.

Μόλις εμφανιστεί η πίστα, ο στόχος είναι η πλοήγηση του παίκτη από την πάνω αριστερή γωνία (εκκίνηση) προς την κάτω δεξιά. Η έξοδος είναι ευδιάκριτη, καθώς αναβοσβήνει και λάμπει περισσότερο από τα υπόλοιπα στοιχεία. Στην πίστα υπάρχουν διάφορα φωτιζόμενα κουτάκια που λειτουργούν ως εμπόδια, τα οποία ο παίκτης πρέπει να αποφύγει.

Η δυσκολία του παιχνιδιού αυξάνεται σταδιακά. Ενώ ο χρόνος είναι πάντα περιορισμένος, όσο ο παίκτης προχωρά σε επόμενα επίπεδα, ο διαθέσιμος χρόνος μειώνεται, απαιτώντας γρηγορότερους και ακριβέστερους χειρισμούς.

## 2.3 Τεχνική Αναφορά

### 2.3.1 Αρχικοποίηση Παραμέτρων



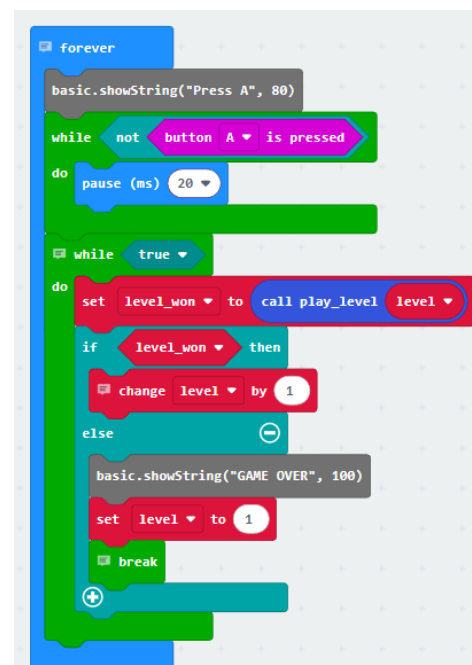
Το μπλοκ on start εκτελείται μία φορά κατά την εκκίνηση του micro:bit και χρησιμεύει για την αρχικοποίηση του παιχνιδιού. Αρχικά, ορίζει τις σταθερές φωτεινότητας: PLAYER\_BRIGHT στο μέγιστο (255) ώστε ο παίκτης να ξεχωρίζει, WALL\_BRIGHT (100) για τους τοίχους, και δύο τιμές για την έξοδο (EXIT\_BRIGHT και EXIT\_DIM) ώστε να μπορεί να αναβοσβήνει. Στη συνέχεια, ρυθμίζει τις παραμέτρους του παιχνιδιού: το TILT\_THRESHOLD (300) καθορίζει την ελάχιστη κλίση που απαιτείται για την κίνηση, ενώ το MOVE\_DELAY (300) προσθέτει μια μικρή καθυστέρηση μεταξύ των κινήσεων για καλύτερο έλεγχο. Τέλος, εμφανίζει τον τίτλο "LABYRINTHOS" στην οθόνη και θέτει το αρχικό επίπεδο (level) στο 1.

### 2.3.2 Κεντρικός Βρόχος και Ροή Παιχνιδιού

Το μπλοκ forever αποτελεί τον κεντρικό βρόχο ελέγχου. Αμέσως μετά την αρχικοποίηση, το πρόγραμμα μπαίνει σε αυτόν τον βρόχο, εμφανίζοντας αρχικά το μήνυμα "Press A" για να καλέσει τον παίκτη να ξεκινήσει.

Χρησιμοποιώντας έναν βρόχο while, το πρόγραμμα "παγώνει" σε κατάσταση αναμονής, εκτελώντας μόνο μια μικρή παύση 20ms, μέχρι να ανιχνεύσει ότι ο παίκτης έχει πατήσει το κουμπί A.

Μόλις ο παίκτης πατήσει το A, το πρόγραμμα εισέρχεται στον κύριο βρόχο παιχνιδιού (while true), ο οποίος διαχειρίζεται τους γύρους. Σε κάθε επανάληψη, καλεί τη συνάρτηση play\_level (η οποία περιέχει όλη τη λογική του παιχνιδιού



για έναν γύρο) και αποθηκεύει το αποτέλεσμα της—true για νίκη ή false για ήττα—στη μεταβλητή `level_won`.

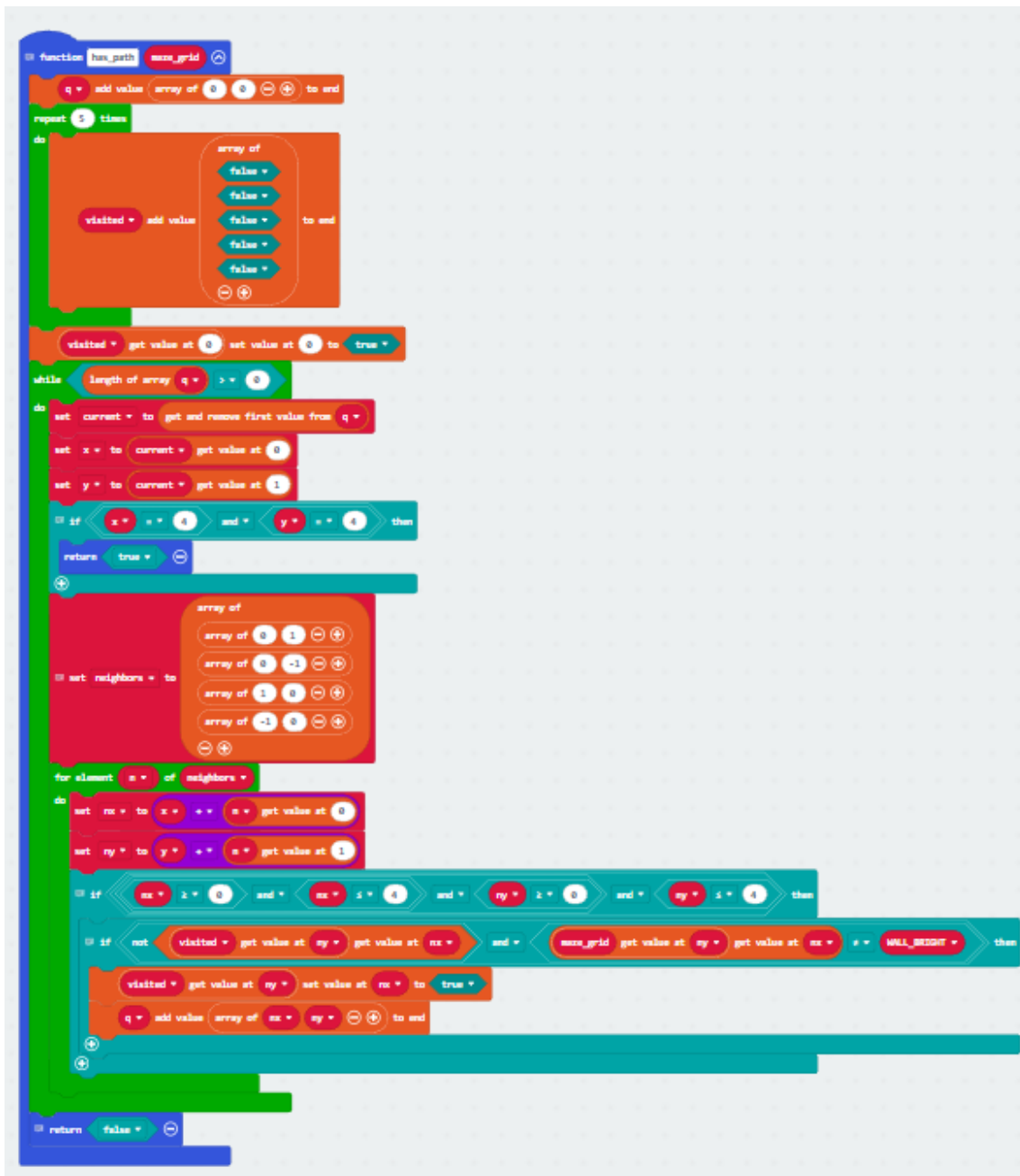
Ανάλογα με την τιμή αυτής της μεταβλητής, το πρόγραμμα αποφασίζει την επόμενη ενέργεια. Αν ο παίκτης νίκησε (`level_won` είναι true), το πρόγραμμα απλώς αυξάνει τη μεταβλητή `level` κατά 1 και ο βρόχος συνεχίζεται, ξεκινώντας έτσι το επόμενο, δυσκολότερο επίπεδο. Αν όμως ο παίκτης έχασε (`level_won` είναι false), το πρόγραμμα εμφανίζει το μήνυμα "GAME OVER", επαναφέρει το `level` στο 1, και εκτελεί `break` για να "σπάσει" τον εσωτερικό βρόχο. Αυτή η διακοπή το επαναφέρει στην αρχική κατάσταση του `forever`, δηλαδή στην οθόνη "Press A", περιμένοντας να ξεκινήσει ένα εντελώς νέο παιχνίδι.

### 2.3.3 Αλγόριθμος Ελέγχου Επίλυσης Λαβυρίνθου

Η συνάρτηση `has_path` (έχει μονοπάτι) είναι ένας "έξυπνος" βοηθητικός αλγόριθμος που ελέγχει αν ο λαβύρινθος (`maze_grid`) που δημιουργήθηκε τυχαία, είναι επίλυσιμος. Ο ρόλος της είναι να επιβεβαιώσει ότι υπάρχει τουλάχιστον μία έγκυρη διαδρομή από την αρχή (πάνω αριστερά, 0,0) μέχρι την έξοδο (κάτω δεξιά, 4,4), χωρίς να περνάει μέσα από τοίχους.

Για να το πετύχει αυτό, χρησιμοποιεί μια μέθοδο αναζήτησης. Αρχικά, δημιουργεί μια "ουρά" (`q`) με συντεταγμένες που πρέπει να ελέγξει, βάζοντας πρώτο το (0,0). Ξεκινά έναν βρόχο που εκτελείται όσο υπάρχουν στοιχεία στην ουρά. Σε κάθε επανάληψη, παίρνει την πρώτη συντεταγμένη από την ουρά. Αν αυτή η συντεταγμένη είναι η έξοδος (4,4), σημαίνει ότι βρήκε μονοπάτι και επιστρέφει αμέσως true.

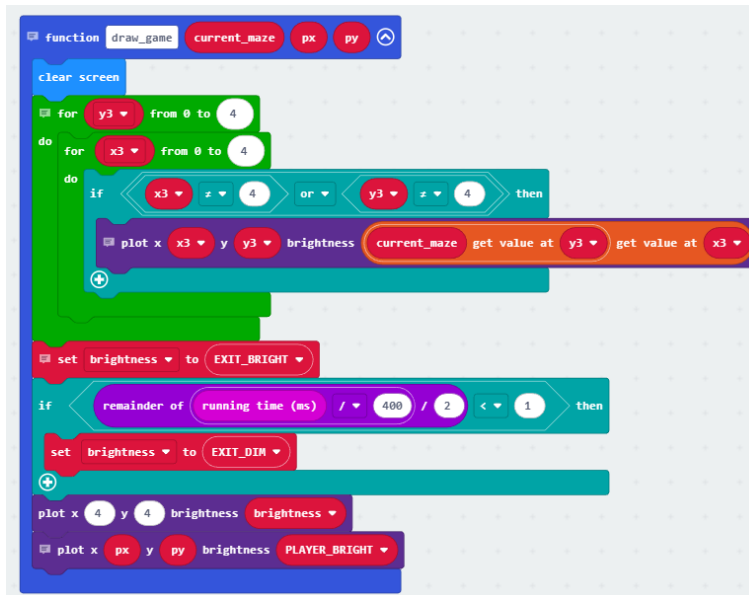
Αν δεν είναι η έξοδος, ελέγχει όλους τους "γείτονες" (πάνω, κάτω, αριστερά, δεξιά). Για κάθε γείτονα, ελέγχει τρία πράγματα: αν είναι εντός των ορίων του πλέγματος (0-4), αν δεν τον έχει ήδη επισκεφθεί, και αν δεν είναι τοίχος (`WALL_BRIGHT`). Αν ένας γείτονας είναι έγκυρος, τον προσθέτει στην ουρά για να ελεγχθεί αργότερα και τον μαρκάρει ως "επισκεφθέντα". Αν ο βρόχος τελειώσει (αδειάσει η ουρά) χωρίς να βρεθεί η έξοδος, σημαίνει ότι δεν υπάρχει λύση, και η συνάρτηση επιστρέφει false.



### 2.3.4 Απεικόνιση Παιχνιδιού

Η συνάρτηση `draw_game` είναι υπεύθυνη για ό,τι βλέπει ο παίκτης στην οθόνη LED 5x5 και εκτελείται συνεχώς για να ανανεώνει την εικόνα. Η διαδικασία ξεκινά πάντα καθαρίζοντας πλήρως την οθόνη. Έπειτα, χρησιμοποιούνται δύο ένθετοι βρόχοι (έναν για το 'x' και έναν για το 'y') για να "σαρώσουν" όλο το πλέγμα. Για κάθε τετράγωνο, η διαδικασία διαβάζει την τιμή του από τον πίνακα `current_maze` και

σχεδιάζει τους τοίχους. Αμέσως μετά, διαχειρίζεται την έξοδο (στο 4,4), χρησιμοποιώντας τον τρέχοντα χρόνο (running time), αλλάζει τη φωτεινότητά της



μεταξύ EXIT\_BRIGHT και EXIT\_DIM κάθε 400ms, δημιουργώντας το χαρακτηριστικό εφέ που αναβοσβήνει. Τέλος, αφού έχει σχεδιάσει τον λαβύρινθο και την έξοδο, σχεδιάζει τον παίκτη (px, py) με την υψηλότερη φωτεινότητα (PLAYER\_BRIGHT), εξασφαλίζοντας ότι είναι πάντα ορατός πάνω από όλα τα άλλα στοιχεία.

### 2.3.5 Συνάρτηση Εκτέλεσης Επιπέδου

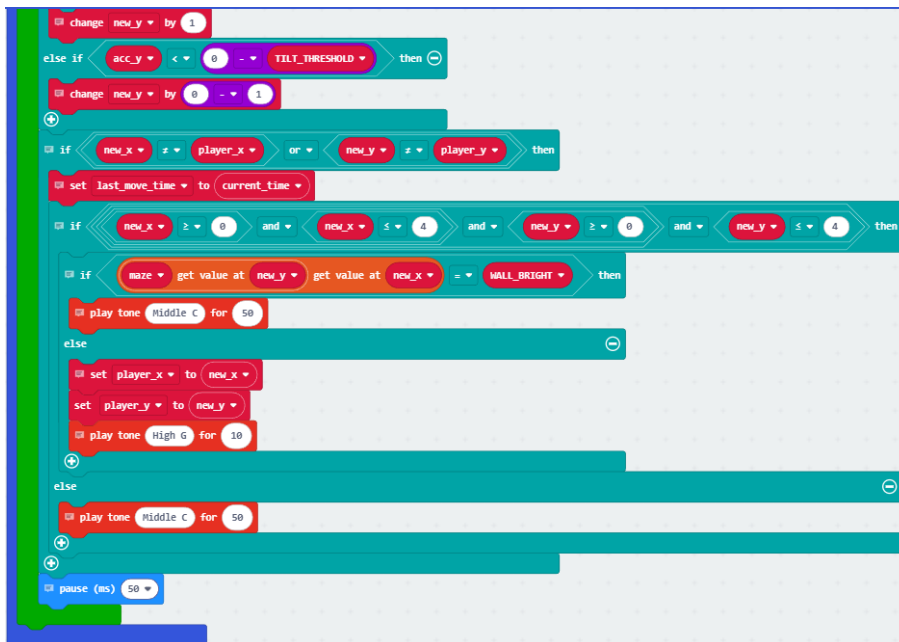
Η συνάρτηση play\_level αποτελεί τον κεντρικό πυρήνα του παιχνιδιού, καθώς εμπεριέχει ολόκληρη τη λογική που εκτελείται κατά τη διάρκεια ενός γύρου. Όταν καλείται από τον forever, η συνάρτηση ξεκινά εμφανίζοντας τον αριθμό του τρέχοντος επιπέδου (lvl) και παίζοντας έναν ήχο έναρξης. Αμέσως μετά, αναλαμβάνει τις αρχικές ρυθμίσεις του γύρου: καλεί τη generate\_maze για να δημιουργήσει μια νέα, έγκυρη πίστα, τοποθετεί τον παίκτη στην αρχική θέση (0,0) και υπολογίζει το χρονικό όριο. Αυτό το όριο είναι δυναμικό, καθώς μειώνεται κατά 2 δευτερόλεπτα για κάθε επίπεδο (με ελάχιστο τα 10 δευτερόλεπτα), αυξάνοντας έτσι την πρόκληση.

Μόλις ολοκληρωθεί η ρύθμιση, η συνάρτηση εισέρχεται στον κύριο βρόχο while true, ο οποίος επαναλαμβάνεται συνεχώς μέχρι ο παίκτης να κερδίσει ή να χάσει. Σε κάθε επανάληψη, το πρόγραμμα ελέγχει πρώτα την συνθήκη ήττας: εάν ο χρόνος που έχει περάσει από την έναρξη του γύρου ξεπεράσει το time\_limit, η συνάρτηση παίζει έναν ήχο αποτυχίας και επιστρέφει την τιμή false, σηματοδοτώντας στο forever ότι το παιχνίδι τελείωσε με ήττα. Εάν ο χρόνος δεν έχει λήξει, η συνάρτηση καλεί το draw\_game για να ανανεώσει την οθόνη, σχεδιάζοντας τον λαβύρινθο και

```
function play_level lvl
  show number lvl
  start melody power up repeating once
  pause (ms) 1000
  set maze to call generate_maze lvl
  set player_x to 0
  set player_y to 0
  set time_limit to max of 10000 and 30000 - lvl * 2000
  set start_time to running time (ms)
  while true
    do
      set current_time to running time (ms)
      if current_time - start_time > time_limit then
        show icon
        start melody waaaaaaa repeating once
        pause (ms) 2000
        return false
      call draw_game maze player_x player_y
      if player_x == 4 and player_y == 4 then
        show icon
        start melody ba ding repeating once
        pause (ms) 2000
        return true
      if current_time - last_move_time < MOVE_DELAY then
        continue
      set acc_x to acceleration (mg) x
      set acc_y to acceleration (mg) y
      set new_x to player_x
      set new_y to player_y
      if acc_x > TILT_THRESHOLD then
        change new_x by 1
      else if acc_x < 0 - TILT_THRESHOLD then
        change new_x by 0 - 1
      if acc_y > TILT_THRESHOLD then
        change new_y by 1
      else if acc_y < 0 - TILT_THRESHOLD then
        change new_y by 0 - 1
```

τον παίκτη. Αμέσως μετά, ελέγχει την συνθήκη νίκης, αν οι συντεταγμένες του παίκτη είναι (4,4), παίζει τον ήχο νίκης και **επιστρέφει true**, ενημερώνοντας το forever για την επιτυχία.

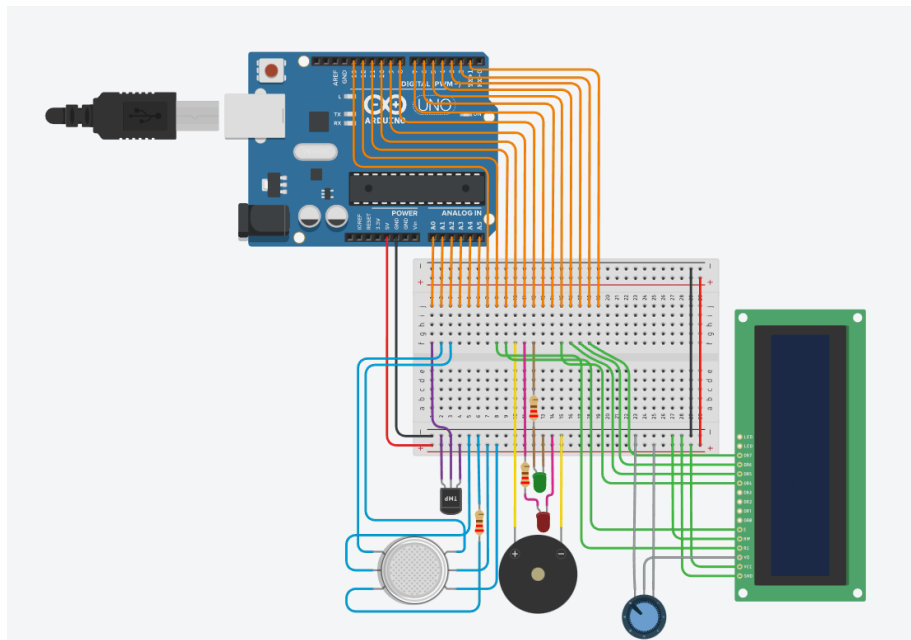
Εάν ο παίκτης ούτε έχει χάσει ούτε έχει κερδίσει, η συνάρτηση προχωρά στον **έλεγχο χειρισμού**. Διαβάζει τους αισθητήρες κλίσης (tilt) του micro:bit. Εάν η κλίση σε κάποιον άξονα ξεπεράσει το καθορισμένο όριο (TILT\_THRESHOLD), το πρόγραμμα υπολογίζει την επόμενη πιθανή θέση του παίκτη. Πριν όμως μετακινήσει τον παίκτη, εκτελεί **έλεγχο σύγκρουσης**: διασφαλίζει ότι η νέα θέση είναι εντός των ορίων του πλέγματος (0-4) και, το κυριότερο, ελέγχει αν στη θέση αυτή υπάρχει τοίχος (WALL\_BRIGHT). Εάν είναι τοίχος, παίζεται ένας ήχος "χτυπήματος" και η κίνηση ακυρώνεται. Εάν η κίνηση είναι έγκυρη, η θέση του παίκτη ενημερώνεται, παίζεται ένας σύντομος ήχος "κίνησης", και ο βρόχος συνεχίζεται.



## 3<sup>η</sup> εργασία: Tinkercad

### 3.1 Λύση

Link: <https://www.tinkercad.com/things/fhlx7lpCcwG-stem-exercise-2025-e22001?sharecode=rwj3WBqzF0WQhOxj48pMxqKDvVpur4U-we3BfNRyx6A>

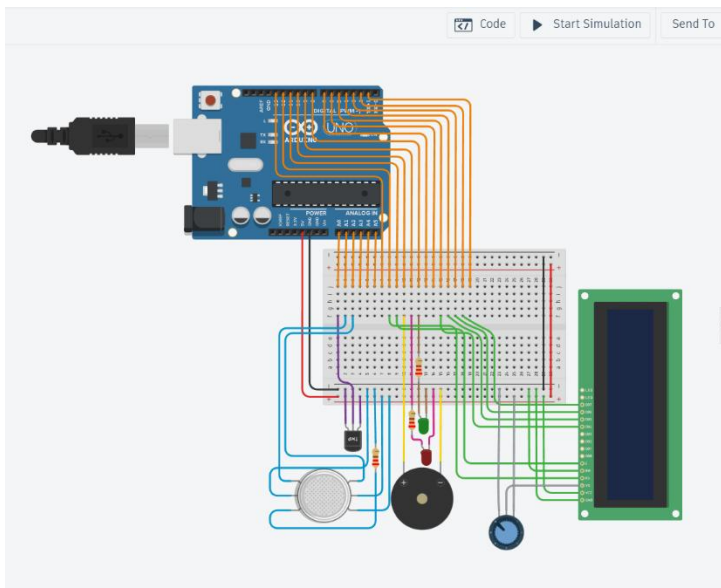




### 3.2 Εκφώνηση

“Σας ζητείται η ανάπτυξη ενός συστήματος συναγερμού που προστατεύει ένα εσωτερικό χώρο από τον κίνδυνο πυρκαγιάς. Το σύστημα θα μετράει τη θερμοκρασία καθώς και θα ανιχνεύει την ύπαρξη καπνού με τη βοήθεια κατάλληλων αισθητήρων. Στην περίπτωση που καταγραφεί υπερβολική θερμοκρασία ( $> 50^{\circ}\text{C}$ ) ή υψηλή συγκέντρωση καπνού, τότε το σύστημα θα πρέπει να ενεργοποιήσει μια σειρήνα, η οποία θα ηχεί σε μοτίβο 500 ms ON/ 500 ms OF, και να ανάψει ένα κόκκινο στοιχείο LED ως ένδειξη της επικινδυνότητας. Επιπλέον, θα υπάρχει πλήκτρο reset για την επαναρχικοποίηση του συναγερμού που θα έχει ως αποτέλεσμα την απενεργοποίηση της σειρήνας και την ενεργοποίηση ενός πράσινου στοιχείου LED που θα υποδεικνύει ότι ο χώρος είναι και πάλι ασφαλής. Παράλληλα με τις υπόλοιπες ενδείξεις που έχουν ήδη αναφερθεί, θα πρέπει το σύστημα να χρησιμοποιεί οθόνη LCD στην οποία θα εμφανίζονται όλες οι καταστάσεις του. Συγκεκριμένα, όταν έχει εντοπιστεί κίνδυνος, τότε θα πρέπει να αναγράφει στην πρώτη γραμμή το μήνυμα “DANGER: FIRE!” και στη δεύτερη το μήνυμα “Press SAFE”. Στη συνέχεια, όταν πατηθεί το κουμπί reset θα εμφανίζεται το μήνυμα “Alarm Cleared”. Τέλος, οι στάθμες της θερμοκρασίας, του καπνού και της κατάστασης λειτουργίας του συναγερμού (SAFE/ALARM) θα πρέπει να εμφανίζονται περιοδικά στο serial monitor”

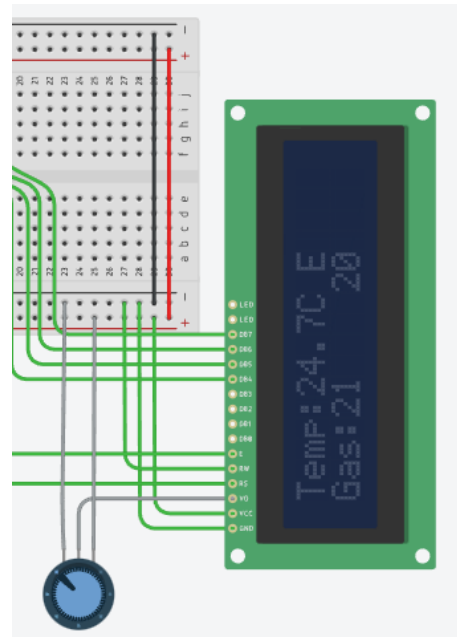
### 3.3 Οδηγίες Χρήσης



Για να ξεκινήσει η προσομοίωση του συστήματος συναγερμού, πρέπει ο χρήστης να πατήσει το κουμπί “Start Simulation” πάνω δεξιά. Θα δει το πράσινο λαμπάκι να ανάβει.

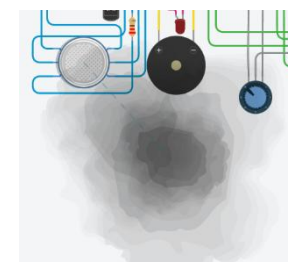
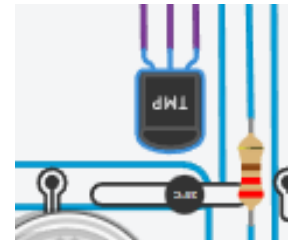


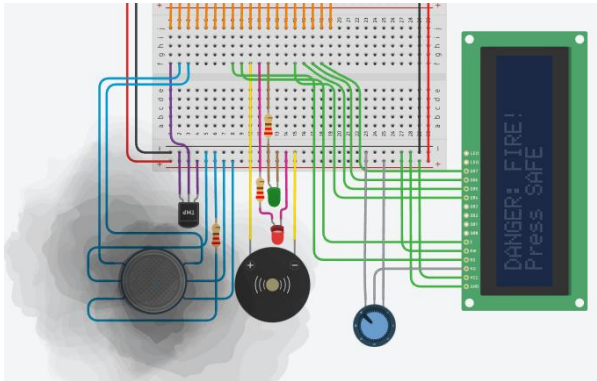
Ύστερα, πρέπει να ανοίξει την LCD οθόνη. Για να το κάνει πρέπει να περιστρέψει δεξιόστροφα το Ποτενσιόμετρο. Ύστερα θα δει την οθόνη να ανάβει και να δείχνει την θερμοκρασία και το επίπεδο καπνού.



Για να αλλάξει ο χρήστης αυτές τις τιμές στην προσπάθεια του να πυροδοτήσει τον συναγερμό, μπορεί είτε:

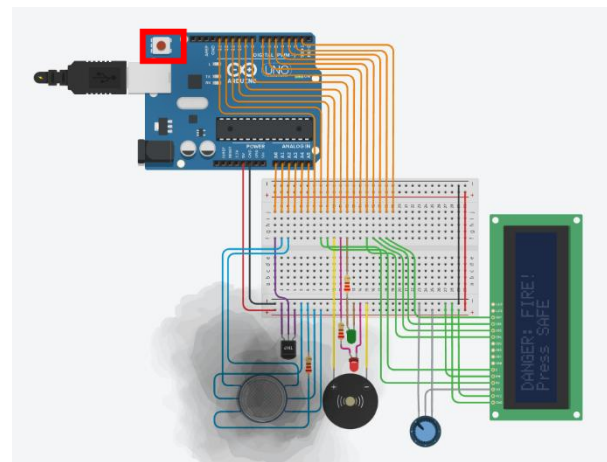
1. **Να αλλάξει την θερμοκρασία:** Πατώντας το θερμόμετρο, μπορεί να αυξομειώσει την θερμοκρασία
2. **Να αλλάξει το επίπεδο καπνού:** Πατώντας τον αισθητήρα καπνού, θα του εμφανιστεί ένα σύννεφο το οποίο μπορεί να μετακινήσει





Θα καταλάβει ότι ο συναγερμός πυροδοτήθηκε, όταν δει το κόκκινο λαμπάκι LED να ανάβει και το buzzer αρχίσει και χτυπάει.

Για να σταματήσει ο συναγερμός πρέπει ο χρήστης να πατήσει το κόκκινο κουμπί που βρίσκεται πάνω στην πλακέτα Arduino Uno.



### 3.4 Συνδεσμολογία

Για την ορθή και οργανωμένη συνδεσμολογία του κυκλώματος, κρίθηκε απαραίτητη η χρήση breadboard (δοκιμαστικής πλακέτας). Ως θεμελιώδες βήμα για τη δομή του κυκλώματος, σύνδεσα τις κύριες γραμμές τροφοδοσίας. Συγκεκριμένα, η έξοδος 5V [από την πλακέτα Arduino/πηγή] συνδέθηκε στη θετική γραμμή (+) του breadboard, ενώ η γείωση (GND) συνδέθηκε στην αρνητική γραμμή (-).

Αυτή η τεχνική εξασφαλίζει πολλαπλά και εύκολα προσβάσιμα σημεία τροφοδοσίας και γείωσης για όλα τα εξαρτήματα, συμβάλλοντας σε ένα πιο τακτοποιημένο, λειτουργικό και ευανάγνωστο κύκλωμα.

Ύστερα, για την ολοκλήρωση της οργάνωσης, σύνδεσα (Από το Arduino στο Breadboard):

|         |          |          |          |           |
|---------|----------|----------|----------|-----------|
| A0 → 1j | A1 → 2j  | A2 → 3j  | A3 → 4j  | A4 → 5j   |
| A5 → 6j | D13 → 7j | D12 → 8j | D11 → 9j | D10 → 10j |



|          |          |          |          |          |
|----------|----------|----------|----------|----------|
| D9 → 11j | D8 → 12j | D7 → 13j | D6 → 14j | D5 → 15j |
| D4 → 16j | D3 → 17j | D2 → 18j | D1 → 19j |          |

Με αυτή τη μέθοδο, οι στήλες 1 έως 19 του breadboard λειτουργούν πλέον ως απευθείας προέκταση των αντίστοιχων ακροδεκτών (pins) του Arduino. Οπότε πλέον μπορούμε να συνδέσουμε όλα τα απαραίτητα components για τον συναγερμό με αυτόν τον τρόπο:

| Αισθητήρας<br>TMP36                | GAS sensor                                                                                           | Buzzer                           | Green LED                                                        |
|------------------------------------|------------------------------------------------------------------------------------------------------|----------------------------------|------------------------------------------------------------------|
| Vout → 1f<br>Vcc → 5V<br>GND → GND | H1 → 5V<br>H2 → GND<br>B1 → 2f<br>B2 → GND (με<br>ενδιάμεσα<br>αντίσταση 10κΩ)<br>A1 → 3f<br>A2 → 5V | Positive → 10f<br>Negative → GND | Anode → 12f (με<br>ενδιάμεσα<br>αντίσταση 220Ω)<br>Cathode → GND |

| Red LED                                                          | LCD                                                                                                                                                     | Ποτενσιόμετρο                                         | Pushbutton             |
|------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------|------------------------|
| Anode → 11f (με<br>ενδιάμεσα<br>αντίσταση 220Ω)<br>Cathode → GND | GND → GND<br>VCC → 5V<br>V0 → μεσαίο πόδι<br>του<br>ποτενσιόμετρου<br>RS → 8f<br>RW → GND<br>E → 9f<br>DB4 → 15f<br>DB5 → 16f<br>DB6 → 17f<br>DB7 → 18f | Terminal1 → 5V<br>Terminal2 → GND<br>Wiper V0 → (LCD) | Δεν<br>χρησιμοποιήθηκε |

### 3.5 Τεχνική Αναφορά

```
#include <LiquidCrystal.h>

// Ορισμός ακίδων
const int TEMP_SENSOR = A0;
const int GAS_SENSOR = A1;
const int BUZZER = 10;
const int BUTTON = 7;
const int GREEN_LED = 8;
const int RED_LED = 9;

// Ορισμός ακίδων LCD
const int RS = 12, EN = 11, D4 = 5, D5 = 4, D6 = 3, D7 = 2;
LiquidCrystal lcd(RS, EN, D4, D5, D6, D7);

// Μεταβλητές
float temperature = 0;
int gasValue = 0;
int gasBaseline = 0; // Baseline for gas sensor
bool alarmActive = false;
bool buttonState = false;
bool lastButtonState = false;
unsigned long lastBuzzerTime = 0;
bool buzzerState = false;

// Κατώφλια
const float TEMP_THRESHOLD = 50.0;
const int GAS_THRESHOLD = 50;
```

**#include <LiquidCrystal.h>:** Γίνεται εισαγωγή της επίσημης βιβλιοθήκης του Arduino για τον χειρισμό οθονών LCD (Liquid Crystal Display).

**const int ...:** Ορίζονται σταθερές (constants) που αντιστοιχίζουν τις φυσικές ακίδες (pins) του Arduino με τα εξαρτήματα. Για παράδειγμα, ο αισθητήρας θερμοκρασίας συνδέεται στην αναλογική είσοδο A0 και ο αισθητήρας αερίου στην A1. Τα LEDs, το buzzer και το κουμπί συνδέονται στις αντίστοιχες ψηφιακές εξόδους/εισόδους.

**LiquidCrystal lcd(...):** Δημιουργείται ένα αντικείμενο lcd που αρχικοποιεί την επικοινωνία με την οθόνη, δηλώνοντας ποιες ακίδες του Arduino (12, 11, 5, 4, 3, 2) αντιστοιχούν στις ακίδες της LCD (RS, EN, D4-D7).

**Μεταβλητές & Κατώφλια:** Ορίζονται οι ολικές μεταβλητές που θα χρησιμοποιεί όλο το πρόγραμμα.

- temperature, gasValue: Αποθηκεύουν τις τρέχουσες μετρήσεις.

- **gasBaseline:** Αποθηκεύει την "κανονική" τιμή του αέρα (βλέπε `setup()`).
- **alarmActive:** Μια μεταβλητή "σημαία" (flag) τύπου `boolean` (`true/false`) που δείχνει αν ο συναγερμός είναι ενεργός.
- **TEMP\_THRESHOLD** και **GAS\_THRESHOLD:** Αυτά είναι τα όρια ασφαλείας. Ο συναγερμός θα χτυπήσει εάν η θερμοκρασία ξεπεράσει τους 50°C ή εάν η τιμή του αερίου ανέβει κατά 50 μονάδες πάνω από το "κανονικό" επίπεδο (`gasBaseline`).

```
void setup() {  
  // Αρχικοποίηση ακίδων  
  pinMode(BUZZER, OUTPUT);  
  pinMode(BUTTON, INPUT);  
  pinMode(GREEN_LED, OUTPUT);  
  pinMode(RED_LED, OUTPUT);  
  
  // Αρχικοποίηση LCD  
  lcd.begin(16, 2);  
  
  // Αρχικοποίηση Serial Monitor  
  Serial.begin(9600);  
  
  // Calibrate gas sensor  
  calibrateGasSensor();  
  
  // Αρχική κατάσταση - ασφαλής  
  digitalWrite(GREEN_LED, HIGH);  
  digitalWrite(RED_LED, LOW);  
  digitalWrite(BUZZER, LOW);  
  
  // Εμφάνιση αρχικού μηνύματος  
  lcd.setCursor(0, 0);  
  lcd.print("System Ready");  
  lcd.setCursor(0, 1);  
  lcd.print("Calibrating...");  
  
  Serial.println("Fire Alarm System Started");  
  Serial.println("Calibrating gas sensor...");  
  delay(2000); // Allow time for calibration  
  
  lcd.clear();  
  lcd.setCursor(0, 0);  
  lcd.print("Status: SAFE");  
  lcd.setCursor(0, 1);  
  lcd.print("Gas Base: ");  
  lcd.print(gasBaseline);  
}
```

Η συνάρτηση `setup()` εκτελείται μία μόνο φορά κατά την εκκίνηση του Arduino.

**pinMode():** Ορίζει τη λειτουργία της κάθε ακίδας. Τα LEDs και το Buzzer ορίζονται ως `OUTPUT` (εξοδος), ενώ το κουμπί ως `INPUT` (είσοδος). Οι αναλογικές ακίδες (A0, A1) δεν χρειάζονται `pinMode()` όταν χρησιμοποιούνται ως είσοδοι.

**lcd.begin() / Serial.begin():** Ξεκινούν την επικοινωνία με την οθόνη LCD (δηλώνοντας ότι είναι 16x2 χαρακτήρων) και με τον υπολογιστή μέσω της σειριακής θύρας (Serial Monitor) για debugging.

**calibrateGasSensor():** Καλείται η συνάρτηση βαθμονόμησης. Αυτό είναι κρίσιμο, καθώς ο αισθητήρας αερίου πρέπει να

"μάθει" ποια είναι η τιμή του καθαρού αέρα στο δωμάτιο.

**Αρχική Κατάσταση:** Το σύστημα μπαίνει αμέσως σε "ασφαλή" κατάσταση (ανάβει το πράσινο LED, σβήνει το κόκκινο και το buzzer).

**Μηνύματα:** Εμφανίζει μηνύματα εκκίνησης και βαθμονόμησης στην LCD και στο Serial Monitor.

```
void calibrateGasSensor() {  
    long sum = 0;  
    for (int i = 0; i < 100; i++) {  
        sum += analogRead(GAS_SENSOR);  
        delay(10);  
    }  
    gasBaseline = sum / 100;  
  
    Serial.print("Gas Sensor Baseline: ");  
    Serial.println(gasBaseline);  
}  
  
void loop() {  
    // Ανάγνωση αισθητήρων  
    readSensors();  
  
    // Έλεγχος κουμπιού reset  
    checkButton();  
  
    // Έλεγχος συνθηκών συναγερμού  
    checkAlarmConditions();  
  
    // Διαχείριση συναγερμού  
    if (alarmActive) {  
        handleAlarm();  
    } else {  
        handleSafeState();  
    }  
  
    // Εμφάνιση στοιχείων στο Serial Monitor  
    displaySerialInfo();  
  
    delay(100);  
}
```

Αυτή η βοηθητική συνάρτηση `calibrateGasSensor()` μετρά την "τιμή βάσης" του αισθητήρα αερίου. Επειδή οι τιμές των αισθητήρων μπορεί να έχουν μικροδιακυμάνσεις ("θόρυβο"), η συνάρτηση παίρνει 100 μετρήσεις από τον GAS\_SENSOR (με μια μικρή παύση 10ms μεταξύ τους) και υπολογίζει τον μέσο όρο τους. Αυτός ο μέσος όρος αποθηκεύεται στην ολική μεταβλητή `gasBaseline` και αντιπροσωπεύει τον "καθαρό αέρα".

Η `loop()` είναι η κύρια συνάρτηση του προγράμματος και εκτελείται συνεχώς:

**`readSensors()`:** Διαβάζει τις τρέχουσες τιμές από τους αισθητήρες.

**`checkButton()`:** Ελέγχει αν πατήθηκε το

κουμπί reset.

**`checkAlarmConditions()`:** Ελέγχει αν οι τιμές έχουν ξεπεράσει τα κατώφλια ασφαλείας.

**`if (alarmActive)`:** Ελέγχει την κατάσταση. Αν η μεταβλητή `alarmActive` είναι true, καλεί τη συνάρτηση `handleAlarm()`. Αν είναι false, καλεί τη `handleSafeState()`.

**`displaySerialInfo()`:** Στέλνει μια αναφορά κατάστασης στο Serial Monitor.

**`delay(100)`:** Μια μικρή παύση 100ms για να σταθεροποιηθεί ο βρόχος και να μην "πλημμυρίζει" το σύστημα με υπερβολικά γρήγορες μετρήσεις.

```
void checkAlarmConditions() {  
    // Calculate gas difference from baseline  
    int gasDifference = gasValue - gasBaseline;  
  
    if (temperature > TEMP_THRESHOLD || gasDifference > GAS_THRESHOLD) {  
        if (!alarmActive) {  
            alarmActive = true;  
            lcd.clear();  
            lcd.setCursor(0, 0);  
            lcd.print("DANGER: FIRE!");  
            lcd.setCursor(0, 1);  
            lcd.print("Press SAFE");  
  
            Serial.println("!!! ALARM TRIGGERED !!!");  
            Serial.print("Temperature: "); Serial.println(temperature);  
            Serial.print("Gas Level: "); Serial.println(gasValue);  
            Serial.print("Gas Difference: "); Serial.println(gasDifference);  
        }  
    }  
}
```

Αυτή είναι η λογική πυροδότησης.

- Υπολογίζει τη διαφορά (gasDifference) της τρέχουσας τιμής αερίου από την τιμή βάσης (καθαρός αέρας).
- Ελέγχει αν η θερμοκρασία είναι πάνω από το όριο || (H) η διαφορά του αερίου είναι πάνω από το όριο.
- Αν μία από τις δύο συνθήκες ισχύει, ελέγχει αν ο συναγερμός δεν είναι ήδη ενεργός (!alarmActive). Αυτό γίνεται για να ενεργοποιηθεί ο συναγερμός μόνο την πρώτη φορά που θα ανιχνευθεί κίνδυνος.
- Αν όλα ισχύουν, θέτει το alarmActive = true και τυπώνει το μήνυμα κινδύνου στην LCD.

```
void handleAlarm() {  
    // Έλεγχος LED  
    digitalWrite(GREEN_LED, LOW);  
    digitalWrite(RED_LED, HIGH);  
  
    // Διαχείριση buzzer (500ms ON/OFF)  
    unsigned long currentTime = millis();  
    if (currentTime - lastBuzzerTime >= 500) {  
        buzzerState = !buzzerState;  
        digitalWrite(BUZZER, buzzerState);  
        lastBuzzerTime = currentTime;  
    }  
}  
  
void handleSafeState() {  
    // Απενεργοποίηση συναγερμού  
    digitalWrite(GREEN_LED, HIGH);  
    digitalWrite(RED_LED, LOW);  
    digitalWrite(BUZZER, LOW);  
  
    // Εμφάνιση μηνύματος ασφαλείας στο LCD  
    lcd.setCursor(0, 0);  
    lcd.print("Temp:");  
    lcd.print(temperature, 1);  
    lcd.print("C ");  
    lcd.setCursor(0, 1);  
    lcd.print("Gas:");  
    lcd.print(gasValue);  
    lcd.print(" ");  
}
```

Η συνάρτηση `handleAlarm()` εκτελείται συνεχώς όταν τα πράγματα είναι "ασφαλή" (`alarmActive == false`). Απλά ανάβει το πράσινο LED, σβήνει τα άλλα, και εμφανίζει τις τρέχουσες μετρήσεις θερμοκρασίας και αερίου στην LCD.

Η συνάρτηση `handleSafeState()` εκτελείται συνεχώς όταν `alarmActive == true`.

- Σβήνει το πράσινο LED και ανάβει το κόκκινο.
- Διαχείριση Buzzer (Non-Blocking): Χρησιμοποιεί τη συνάρτηση `millis()` για να διαχειριστεί το buzzer χωρίς να "παγώνει" τον κώδικα (όπως θα έκανε ένα `delay()`). Ελέγχει αν έχουν περάσει

500ms από την τελευταία αλλαγή. Αν ναι, αντιστρέφει την κατάσταση του buzzer (από HIGH σε LOW ή το ανάποδο), πετυχαίνοντας έτσι τον διακοπτόμενο ήχο "μπιπ-μπιπ".

Η συνάρτηση `resetAlarm()` εκτελείται όταν πατηθεί το κουμπί ενώ ο συναγερμός χτυπά.

1. Θέτει το `alarmActive = false` για να σταματήσει ο συναγερμός.
2. Εμφανίζει μήνυμα στην LCD ότι ο συναγερμός μηδενίστηκε.
3. Καλεί ξανά τη συνάρτηση **`calibrateGasSensor()`**. Αυτό είναι απαραίτητο διότι, μετά από ένα συμβάν καπνού, ο "καθαρός αέρας" μπορεί να έχει αλλάξει. Το σύστημα πρέπει να μάθει τη νέα τιμή βάσης.
4. Επιστρέφει στην κανονική λειτουργία καλώντας τη `handleSafeState()`.

```
void resetAlarm() {  
    alarmActive = false;  
    lcd.clear();  
    lcd.setCursor(0, 0);  
    lcd.print("Alarm Cleared ");  
    delay(2000);  
  
    // Recalibrate gas sensor after alarm  
    calibrateGasSensor();  
  
    // Επιστροφή σε κανονική λειτουργία  
    lcd.clear();  
    handleSafeState();  
}
```





Παρόμοια με το buzzer, αυτή η συνάρτηση χρησιμοποιεί `millis()` για να στέλνει μια πλήρη αναφορά κατάστασης (θερμοκρασία, αέριο, τιμή βάσης, κατάσταση συναγερμού) στο Serial Monitor του υπολογιστή κάθε 2 δευτερόλεπτα, χωρίς να μπλοκάρει τον κύριο βρόχο `loop()`.

```
void displaySerialInfo() {  
    static unsigned long lastSerialTime = 0;  
    if (millis() - lastSerialTime > 2000) {  
        int gasDifference = gasValue - gasBaseline;  
  
        Serial.println("=== System Status ===");  
        Serial.print("Temperature: ");  
        Serial.print(temperature);  
        Serial.println(" °C");  
  
        Serial.print("Gas Level: ");  
        Serial.print(gasValue);  
        Serial.print(" (Baseline: ");  
        Serial.print(gasBaseline);  
        Serial.print(", Difference: ");  
        Serial.print(gasDifference);  
        Serial.println(")");  
  
        Serial.print("Alarm Status: ");  
        if (alarmActive) {  
            Serial.println("ALARM - DANGER!");  
        } else {  
            Serial.println("SAFE");  
        }  
  
        Serial.print("Thresholds - Temp: ");  
        Serial.print(TEMP_THRESHOLD);  
        Serial.print("°C, Gas Diff: ");  
        Serial.println(GAS_THRESHOLD);  
        Serial.println("=====");  
  
        lastSerialTime = millis();  
    }  
}
```